AJAX

AJAX (Asynchronous JavaScript and XML) is a web development technique that allows client-side scripts (JavaScript) to communicate with a server without reloading the entire web page. This creates faster, more dynamic user experiences.

It is a **technique** in JavaScript that allows a webpage to:
- send data to the server
- fetch data from the server
- update parts of the webpage
- without reloading the entire webpage

Examples of AJAX in real websites:

- Search suggestions (Google)
- Autocomplete in forms
- Submitting forms without page reload
- Loading more posts (Facebook "See more")
- Live notifications

How AJAX Works

AJAX uses JavaScript to create background HTTP requests to the server.

Flow Diagram:

```
                JavaScript triggers AJAX
                          ↓
        Browser sends HTTP request (XHR / Fetch)
                          ↓
                Server processes request
                          ↓
          Server sends response (JSON/Text/XML)
                          ↓
              JavaScript receives response
                          ↓
           Page updates dynamically (no reload)
```

AJAX is implemented in JavaScript mainly using:

- **XMLHttpRequest (Old but important)**

- **Fetch API (Modern)**

Why is AJAX Asynchronous?

"Asynchronous" means:

- JavaScript does **not wait** for the server response
- Page continues working
- When the response arrives, JavaScript handles it

This prevents the UI from freezing.

The Core Component of AJAX: XMLHttpRequest

XMLHttpRequest (XHR) is a built-in JavaScript object used to perform:

- GET
- POST
- PUT
- DELETE

  HTTP requests **in the background**.

Create:

```
const xhr = new XMLHttpRequest();
```

AJAX With XMLHttpRequest

AJAX request always follows 5 steps:

```
// 1. Create request object
const xhr = new XMLHttpRequest();

// 2. Open the request
xhr.open("GET", "https://api.example.com/data");

// 3. Set headers (Optional)
xhr.setRequestHeader("Content-Type", "application/json");
```

```
// 4. Attach event listeners (response handling)
xhr.onload = function() {
  // response handling
};

// 5. Send request
xhr.send();
```

Understanding `readyState` in AJAX

XHR uses `readyState` to tell JavaScript the request progress.

Value	State	Meaning
0	UNSENT	XHR object created
1	OPENED	`.open()` called
2	HEADERS_RECEIVED	Server acknowledged
3	LOADING	Response downloading
4	DONE	Request finished

Example:

```
xhr.onreadystatechange = function() {
  if (xhr.readyState === 4) {
    console.log("Full response received");
  }
};
```

AJAX GET Request

GET = retrieve data from the server.

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "https://jsonplaceholder.typicode.com/posts");

xhr.onload = function() {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    console.log("Received:", data);
  }
};

xhr.send();
```

What happens?

- JavaScript sends request to server
- Server returns JSON
- **`xhr.responseText`** contains string JSON
- **`JSON.parse()`** converts it to a JS object

AJAX POST Request (Send Data to Server)

```
const xhr = new XMLHttpRequest();
xhr.open("POST", "https://jsonplaceholder.typicode.com/posts");
xhr.setRequestHeader("Content-Type", "application/json");

xhr.onload = function () {
  console.log("Response:", JSON.parse(xhr.responseText));
};
const body = {
  title: "New Post",
  body: "Hello",
  userId: 1
};
xhr.send(JSON.stringify(body));
```

What happens?

- Server receives JSON data
- Response includes new created record
- Status code often **201 (Created)**

AJAX PUT Request (Update Data)

```
const xhr = new XMLHttpRequest();
xhr.open("PUT", "https://jsonplaceholder.typicode.com/posts/1");
xhr.setRequestHeader("Content-Type", "application/json");

xhr.onload = function() {
  console.log("Updated:", xhr.responseText);
};

xhr.send(JSON.stringify({
  title: "Updated Title",
  body: "Updated Body"
}));
```

AJAX DELETE Request

```
const xhr = new XMLHttpRequest();
xhr.open("DELETE", "https://jsonplaceholder.typicode.com/posts/1");
xhr.onload = function() {
  console.log("Deleted:", xhr.status);
};

xhr.send();
```

Handling Errors in AJAX

Network Error

```
xhr.onerror = function() {
  console.log("Network error!");
};
```

Timeout Error

```
xhr.timeout = 5000;
xhr.ontimeout = function() {
  console.log("Request timed out");
};
```

Handling Different Response Types

String response

```
xhr.responseText;
```

JSON response

```
JSON.parse(xhr.responseText);
```

XML response

```
xhr.responseXML;
```

AJAX Progress Tracking

Track download:

```
xhr.onprogress = function(e) {
  console.log(`Downloaded: ${e.loaded} bytes`);
};
```

Track upload:

```
xhr.upload.onprogress = function(e) {
  console.log(`Uploaded: ${e.loaded} bytes`);
};
```

AJAX vs Traditional Form Submission

Traditional	AJAX
Page reloads	No reload
Slower	Fast
Poor UX	Great UX
Blocking	Non-blocking

Mistake	Correct Approach	Explanation
Forget to parse JSON	Use **`JSON.parse()`** to convert response text into an object	**`XMLHttpRequest`** returns **string** data, so you must parse it before using
Using synchronous XHR (`false` in open())	Always make requests **asynchronous** (**`true`** in open())	Synchronous XHR blocks the entire UI thread and freezes the browser
Not handling errors	Add **`.onerror`** or check **`status`** for non-200 codes	Prevents silent failures and helps students debug network issues
Setting headers before **`open()`**	Call **`xhr.setRequestHeader()`** **after `xhr.open()`**	Headers can only be set once the request has been initialized

Example (Rendering HTML)

HTML:

```
<div id="output"></div>
```

JavaScript:

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "https://jsonplaceholder.typicode.com/users");

xhr.onload = function () {
  const users = JSON.parse(xhr.responseText);
  let html = "";
  users.forEach(user => {
    html += `
      <h3>${user.name}</h3>
      <p>Email: ${user.email}</p>
      <hr>
```

```
    `;
  });

  document.getElementById("output").innerHTML = html;
};

xhr.send();
```

Why Learn AJAX Today?

Even though Fetch API is modern, AJAX (XHR) is still important:

✓ Used in legacy applications
✓ Used in many interview questions
✓ Helps understand HTTP basics
✓ Supports upload progress better
✓ Good for beginners

AJAX Summary

Feature	Description
AJAX	Asynchronous requests
XHR	JavaScript object for HTTP
GET	Fetch data
POST	Send data
PUT	Update data
DELETE	Remove data
responseText	Server response
readyState	Request status
onload	Response handler
onerror	Error handler

Fetch API

The **Fetch API** is a modern and powerful way to make HTTP requests in JavaScript.
It provides a simpler and cleaner syntax compared to older techniques like **XMLHttpRequest**.

Fetch is built on **Promises**, which makes it easier to write readable and cleaner asynchronous code.

Key Features

- Easy to use
- Promise-based (supports async/await)
- Better error handling than XHR
- Supports request configuration (headers, body, methods)
- Works in browsers and modern JS environments

Making Requests with Fetch API

Fetch uses the following basic syntax:

```
fetch("https://api.example.com/data")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error("Error:", error));
```

How it works

1. **`fetch()`** sends a request to the server.
2. **`response`** object returns with status + headers.
3. **`response.json()`** converts JSON text into a JavaScript object.
4. **`.catch()`** handles network or connection errors.

All Fetch Requests

Using `.then()` and `.catch()` for each request.

```
// Base API URL
const API_URL = "https://jsonplaceholder.typicode.com/posts";

// GET Request - Retrieve Data

function getPosts() {
    fetch(API_URL)
        .then(response => response.json())
        .then(data => {
            console.log("GET → All Posts:", data);
        })
        .catch(error => console.error("GET Error:", error));
}

// POST Request - Create New Data

function createPost() {
    fetch(API_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            title: "New Post",
            body: "This is a new post created using Fetch",
            userId: 1
        })
    })
        .then(response => response.json())
        .then(data => {
            console.log("POST → Created Post:", data);
        })
        .catch(error => console.error("POST Error:", error));
}

// PUT Request - Update Entire Record

function updatePost() {
    fetch(`${API_URL}/1`, {
        method: "PUT",
```

```
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            id: 1,
            title: "Updated Title",
            body: "Updated Content",
            userId: 1
        })
    })
        .then(response => response.json())
        .then(data => {
            console.log("PUT → Updated Post:", data);
        })
        .catch(error => console.error("PUT Error:", error));
}

// DELETE Request - Delete Record

function deletePost() {
    fetch(`${API_URL}/1`, {
        method: "DELETE"
    })
        .then(() => {
            console.log("DELETE → Post deleted successfully");
        })
        .catch(error => console.error("DELETE Error:", error));
}

// Call all functions
getPosts();
createPost();
updatePost();
deletePost();
```

All Fetch Requests (async/await)

```
// Base API URL
const API_URL = "https://jsonplaceholder.typicode.com/posts";

//  GET Request - Retrieve Data
async function getPosts() {
    try {
        const response = await fetch(API_URL);
        const data = await response.json();
        console.log("GET → All Posts:", data);
    } catch (error) {
        console.error("GET Error:", error);
    }
}

//  POST Request - Create New Data
async function createPost() {
    try {
        const response = await fetch(API_URL, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({
                title: "New Post",
                body: "This is a new post created using Fetch",
                userId: 1
            })
        });

        const data = await response.json();
        console.log("POST → Created Post:", data);
    } catch (error) {
        console.error("POST Error:", error);
    }
}

//  PUT Request - Update Entire Record
async function updatePost() {
    try {
        const response = await fetch(`${API_URL}/1`, {
```

```
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({
                id: 1,
                title: "Updated Title",
                body: "Updated Content",
                userId: 1
            })
        });

        const data = await response.json();
        console.log("PUT → Updated Post:", data);
    } catch (error) {
        console.error("PUT Error:", error);
    }
}

//  DELETE Request - Delete Record
async function deletePost() {
    try {
        await fetch(`${API_URL}/1`, {
            method: "DELETE"
        });

        console.log("DELETE → Post deleted successfully");
    } catch (error) {
        console.error("DELETE Error:", error);
    }
}

// Call all functions
getPosts();
createPost();
updatePost();
deletePost();
```

Handling Different Response Types

Servers can return different kinds of data.
Fetch can handle:

Response Type	Fetch Method	Example
JSON	`.json()`	API response data
Text	`.text()`	HTML, plain text
Blob	`.blob()`	Images, files
ArrayBuffer	`.arrayBuffer()`	Binary data
FormData	`.formData()`	Form submissions

JSON Response

```
fetch("/api/products")
    .then(res => res.json())
    .then(data => console.log("JSON:", data));
```

Text Response

```
fetch("/message.txt")
    .then(res => res.text())
    .then(text => console.log("TEXT:", text));
```

Blob Response (Images)

```
fetch("/image.png")
    .then(res => res.blob())
    .then(blob => {
        const imgUrl = URL.createObjectURL(blob);
        document.getElementById("photo").src = imgUrl;
    });
```

Error Handling in Fetch

Fetch does **not** throw an error for HTTP status codes like 404 or 500.
You must check the status manually:

```
// Using .then()

fetch("/api/data")
    .then(res => {
        if (!res.ok) {
            throw new Error("HTTP Error: " + res.status);
        }
        return res.json();
    })
    .then(data => console.log(data))
    .catch(err => console.error("Error:", err));

// Using async/await + try...catch

async function load() {
  try {
    const res = await fetch("/api/data");
    if (!res.ok) throw res.status;
    console.log(await res.json());
  } catch (err) {
    console.error("Error:", err);
  }
}
load();
```

Third-Party Libraries for Making HTTP Requests in JavaScript

While JavaScript provides built-in tools like **XMLHttpRequest** and **Fetch API**, many developers use third-party libraries to simplify HTTP communication, improve browser support, reduce boilerplate, and add powerful features like interceptors, retries, and automatic data handling.

Popular Third-Party HTTP Request Libraries

Library	Description
Axios	Modern, promise-based HTTP client used in almost all modern JavaScript frameworks. Supports automatic JSON handling, interceptors, and full browser/Node support.
jQuery AJAX	Part of jQuery library; used for network requests in older websites and legacy enterprise systems. Uses callbacks.
SuperAgent	Lightweight, flexible HTTP client for both Node.js and the browser.
Request (Node.js)	Very popular old Node.js HTTP library; now deprecated.
Got (Node.js)	Modern replacement for Request; easy to use, supports promises.
Ky	Tiny wrapper around Fetch with extra features (timeout, retry).
Needle (Node.js)	Lightweight, focused library for Node.js HTTP requests.
$http (AngularJS)	Legacy AngularJS HTTP module (now replaced by Angular's HttpClient).
Node-Fetch (Node.js)	Fetch API implementation for Node.js environments.
Cross-fetch	Universal Fetch polyfill for both browser and Node.

For the browser, the **two most widely used** are:

- **Axios**
- **jQuery AJAX**

Below is the detailed explanation of both.

Axios

Axios is a popular JavaScript library used to make HTTP requests from both the **browser** and **Node.js**. It is built on top of promises and includes many features that Fetch does not provide by default.

Why Developers Use Axios?

- Automatically converts JSON responses.
- Has built-in error handling.
- Supports request cancellation.
- Allows interceptors (modify requests/responses globally).
- Works on all browsers (including older ones).
- Cleaner, simpler syntax than Fetch.

Axios helps students avoid common mistakes like forgetting to convert JSON or mishandling errors.

Installing Axios

Using CDN (For Browser)

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
```

Using NPM (For Node.js or bundlers like Webpack/Vite)

```
npm install axios
```

Axios All HTTP Requests

This code shows **all major HTTP methods** in one example.

```
// Base URL for API
const API_URL = "https://jsonplaceholder.typicode.com/posts";

// GET Request
axios.get(API_URL)
```

```
    .then(response => {
        console.log("GET Response:", response.data);
    })
    .catch(error => console.error("GET Error:", error));


// POST Request
axios.post(API_URL, {
    title: "New Post",
    body: "This is a test post.",
    userId: 1
})
.then(response => {
    console.log("POST Response:", response.data);
})
.catch(error => console.error("POST Error:", error));


// PUT Request (update full record)
axios.put(`${API_URL}/1`, {
    title: "Updated Title",
    body: "Updated content.",
    userId: 1
})
.then(response => {
    console.log("PUT Response:", response.data);
})
.catch(error => console.error("PUT Error:", error));


// DELETE Request
axios.delete(`${API_URL}/1`)
    .then(response => {
        console.log("DELETE Response:", response.data);
    })
    .catch(error => console.error("DELETE Error:", error));
```

jQuery AJAX

jQuery AJAX is an older but still widely used method for making HTTP requests in many legacy and enterprise web applications. It uses the **`$.ajax()`** function and provides powerful configuration options.

Why jQuery AJAX?

- Extremely flexible
- Works on all browsers, including very old ones
- Provides shortcut methods like **`.get()`**, **`.post()`**
- Offers advanced configurations (timeouts, retries, etc.)

Even though modern apps prefer Fetch or Axios, students should still understand jQuery AJAX as it's used in older codebases.

Include jQuery in Browser

CDN Script

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

No installation needed for browser use.

jQuery AJAX – All HTTP Requests

```
const API_URL = "https://jsonplaceholder.typicode.com/posts";

// GET Request
$.ajax({
    url: API_URL,
    method: "GET",
    success: function (data) {
        console.log("GET Response:", data);
    },
    error: function (err) {
        console.error("GET Error:", err);
```

```
    }
});


// POST Request
$.ajax({
    url: API_URL,
    method: "POST",
    contentType: "application/json",
    data: JSON.stringify({
        title: "New jQuery Post",
        body: "This is created via jQuery.",
        userId: 1
    }),
    success: function (data) {
        console.log("POST Response:", data);
    },
    error: function (err) {
        console.error("POST Error:", err);
    }
});


// PUT Request
$.ajax({
    url: `${API_URL}/1`,
    method: "PUT",
    contentType: "application/json",
    data: JSON.stringify({
        title: "Updated Title",
        body: "Updated content",
        userId: 1
    }),
    success: function (data) {
        console.log("PUT Response:", data);
    },
    error: function (err) {
        console.error("PUT Error:", err);
    }
});


// DELETE Request
```

```
$.ajax({
    url: `${API_URL}/1`,
    method: "DELETE",
    success: function (data) {
        console.log("DELETE Response:", data);
    },
    error: function (err) {
        console.error("DELETE Error:", err);
    }
});
```

jQuery AJAX – Inbuilt Methods

```
// 1. $.ajax()
$.ajax({
    url: "URL",
    method: "GET/POST/PUT/DELETE",
    data: {},
    dataType: "json",
    headers: {},
    success: function (response) {},
    error: function (xhr, status, error) {},
    complete: function () {}
});

// 2. $.get()
$.get("URL", { data }, function (response) {});

// 3. $.post()
$.post("URL", { data }, function (response) {});

// 4. $.getJSON()
$.getJSON("URL", function (data) {});

// 5. $.getScript()
$.getScript("script.js", function () {});

// 6. .load()
$("#element").load("URL", function (response, status) {});
```

Comparison Table: XHR vs Fetch vs Axios vs jQuery AJAX

Feature / Library	XMLHttpRequest (XHR)	Fetch API	Axios	jQuery AJAX
Modern / Legacy	Legacy	Modern	Modern	Older / Legacy
Syntax Style	Verbose, event-based	Promise-based	Promise-based	Callback-based
Ease of Use	Hard	Medium	Very Easy	Medium
Auto JSON Parsing	No (use JSON.parse)	No (must call response.json())	Yes	No
Error Handling	Complex, manual	Basic	Strong built-in error support	Basic
Upload/Download Progress	Yes	Not supported directly	Yes	Yes
Support for older browsers	Strong	Good but not IE	Good	Excellent
Cancel a Request	Difficult	Limited	Easy	Limited
Timeout Handling	Supported	Not built-in	Supported	Supported
Need External Library	No	No	Yes	Yes
Default Headers	None	Minimal	Many helpful defaults	None
Handles FormData	Yes	Yes	Yes	Yes
Best Use Case	Legacy support, progress tracking	Modern apps, simple requests	Modern frameworks & API-heavy apps	Legacy jQuery applications

JSON Server

JSON Server is a tool that instantly creates a complete REST API using a simple JSON file. It requires no backend programming and no database setup. It is useful for frontend developers, students learning APIs, testing applications, and mock server creation.

Key points:

- Reads data from a file called **`db.json`**
- Automatically creates REST API endpoints
- Supports filtering, sorting, pagination, search
- Supports relationships between collections
- Can be used with frontend frameworks like React, Angular, Vue, Next.js

Installation Methods

Two ways to set up JSON Server:

Global Installation (No package.json needed)

Install globally:

```
npm install -g json-server
```

Start server:

```
json-server --watch db.json --port 3000
```

This method is simple and quick.

Local Installation (With package.json)

Initialize a new project:

```
npm init -y
```

Install JSON Server as a development dependency:

```
npm install --save-dev json-server
```

Install axios for sending HTTP requests via JavaScript:

```
npm install axios
```

Add the following scripts inside `package.json`:

```
"scripts": {
  "start:server": "json-server --watch db.json --port=3000",
  "run:requests": "node allRequests.js"
}
```

This method is better for long-term projects and teamwork.

Creating the Database File (db.json)

JSON Server reads its data from a JSON file.
Create a file named **`db.json`** with the following content:

```
{
  "students": [
    { "id": 1, "name": "Amit", "age": 20, "course": "JavaScript" },
    { "id": 2, "name": "Pooja", "age": 22, "course": "React" }
  ],
  "marks": [
    { "id": 1, "studentId": 1, "subject": "Math", "score": 90 },
    { "id": 2, "studentId": 1, "subject": "Science", "score": 88 },
    { "id": 3, "studentId": 2, "subject": "English", "score": 92 }
  ]
}
```

Each key such as **`students`** and **`marks`** is called a "collection".

Each object inside the collection is a record.

Starting the Server

Using global installation:

```
json-server --watch db.json --port 3000
```

Using package.json:

```
npm run start:server
```

The API will be available at:

```
http://localhost:3000
```

API Endpoints Created Automatically

Based on the `"students"` collection, JSON Server generates:

GET /students
GET /students/1
POST /students
PUT /students/1
PATCH /students/1
DELETE /students/1

JSON Server automatically handles IDs, routing, and storage.

Understanding JSON Server Concepts

Collections: The keys in the JSON file are called collections (e.g., students, marks).
Records: Each object inside a collection is a record.
Auto ID: JSON Server automatically increments IDs for new records during POST.
Routing System: Automatically generates full REST routes.
Query System: Supports filters, sorting, pagination, search.
Relationships: Supports linking data using foreign keys such as studentId.

All HTTP Requests Using (Fetch API)

```
// Base URL of JSON Server
const BASE_URL = "http://localhost:3000/students";

function getAllStudents() {
return fetch(BASE_URL)
.then(res => res.json())
.then(data => console.log("GET All:", data));
}

// 2. GET - Fetch Single Student
function getStudentById(id) {
return fetch(`${BASE_URL}/${id}`)
.then(res => res.json())
.then(data => console.log("GET One:", data));
}

// 3. POST - Add Student
function addStudent() {
return fetch(BASE_URL, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ name: "Aman", score: 78 })
})
.then(res => res.json())
.then(data => console.log("POST Added:", data));
}

// 4. PUT - Replace Entire Student Record
function replaceStudent() {
return fetch(`${BASE_URL}/1`, {
method: "PUT",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ name: "Ram Updated", score: 95 })
})
.then(res => res.json())
.then(data => console.log("PUT Updated:", data));
}

// 5. PATCH - Update a Field
```

```
function updateStudentScore() {
return fetch(`${BASE_URL}/2`, {
method: "PATCH",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ score: 92 })
})
.then(res => res.json())
.then(data => console.log("PATCH Updated:", data));
}

// 6. DELETE - Remove Student
function deleteStudent() {
return fetch(`${BASE_URL}/3`, { method: "DELETE" })
.then(() => console.log("DELETE Done"));
}

// Run all operations in sequence
async function runAllRequests() {
await getAllStudents(); // GET all
await getStudentById(1); // GET one
await addStudent(); // POST
await replaceStudent(); // PUT
await updateStudentScore(); // PATCH
await deleteStudent(); // DELETE
}

// Execute
runAllRequests();
```

All HTTP Requests (AJAX / XMLHttpRequest)

```
const BASE_URL = "http://localhost:3000/students";

// Helper function
function ajaxRequest(method, url, data = null) {
return new Promise((resolve, reject) => {
const xhr = new XMLHttpRequest();
xhr.open(method, url);
xhr.setRequestHeader("Content-Type", "application/json");
```

```
xhr.onload = () => {
if (xhr.status >= 200 && xhr.status < 300) {
resolve(xhr.responseText ? JSON.parse(xhr.responseText) : {});
} else {
reject("Error: " + xhr.status);
}
};

xhr.onerror = () => reject("Network error");

xhr.send(data ? JSON.stringify(data) : null);
});
}


// 1. GET All Students
function getAllStudents() {
return ajaxRequest("GET", BASE_URL)
.then(data => console.log("GET All:", data));
}

// 2. GET Single Student
function getStudentById(id) {
return ajaxRequest("GET", `${BASE_URL}/${id}`)
.then(data => console.log("GET One:", data));
}

// 3. POST Add New Student
function addStudent() {
return ajaxRequest("POST", BASE_URL, { name: "Aman", score: 78 })
.then(data => console.log("POST Added:", data));
}

// 4. PUT Replace Entire Student
function replaceStudent() {
return ajaxRequest("PUT", `${BASE_URL}/1`, {
name: "Ram Updated",
score: 95
}).then(data => console.log("PUT Updated:", data));
}

// 5. PATCH Update Student Score
```

```
function updateStudentScore() {
return ajaxRequest("PATCH", `${BASE_URL}/2`, { score: 92 })
.then(data => console.log("PATCH Updated:", data));
}

// 6. DELETE Remove Student
function deleteStudent() {
return ajaxRequest("DELETE", `${BASE_URL}/3`)
.then(() => console.log("DELETE Done"));
}

// Run Everything Step-by-Step
async function runAllRequestsAjax() {
await getAllStudents(); // GET all
await getStudentById(1); // GET one
await addStudent(); // POST
await replaceStudent(); // PUT
await updateStudentScore(); // PATCH
await deleteStudent(); // DELETE
}

// Execute
runAllRequestsAjax();
```

All HTTP Requests Using jQuery AJAX

```
<!DOCTYPE html>
<html>
<head>
    <title>jQuery AJAX CRUD Example</title>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

<script>

// BASE URL
const BASE_URL = "http://localhost:3000/students";
```

```
// Helper Function using jQuery AJAX
function ajaxRequest(method, url, data = null) {
    return $.ajax({
        url: url,
        method: method,
        contentType: "application/json",
        data: data ? JSON.stringify(data) : null
    });
}

// 1. GET All Students
function getAllStudents() {
    return ajaxRequest("GET", BASE_URL)
        .then(data => console.log("GET All:", data))
        .catch(err => console.error(err));
}

// 2. GET Student by ID
function getStudentById(id) {
    return ajaxRequest("GET", `${BASE_URL}/${id}`)
        .then(data => console.log("GET One:", data))
        .catch(err => console.error(err));
}

// 3. POST - Add Student
function addStudent() {
    return ajaxRequest("POST", BASE_URL, {
        name: "Aman",
        score: 78
    })
    .then(data => console.log("POST Added:", data))
    .catch(err => console.error(err));
}

// 4. PUT - Replace Entire Student
function replaceStudent() {
    return ajaxRequest("PUT", `${BASE_URL}/1`, {
        name: "Ram Updated",
        score: 95
    })
    .then(data => console.log("PUT Updated:", data))
    .catch(err => console.error(err));
}
```

```
// 5. PATCH - Update Only Score
function updateStudentScore() {
    return ajaxRequest("PATCH", `${BASE_URL}/2`, {
        score: 92
    })
    .then(data => console.log("PATCH Updated:", data))
    .catch(err => console.error(err));
}

// 6. DELETE - Remove Student
function deleteStudent() {
    return ajaxRequest("DELETE", `${BASE_URL}/3`)
        .then(() => console.log("DELETE Done"))
        .catch(err => console.error(err));
}

// Run All API Calls Step-by-Step
async function runAllRequests() {
    await getAllStudents();        // GET all
    await getStudentById(1);       // GET one
    await addStudent();            // POST
    await replaceStudent();        // PUT
    await updateStudentScore();    // PATCH
    await deleteStudent();         // DELETE
}

runAllRequests();

</script>
</body>
</html>
```

All HTTP Requests Using Axios

```
<!DOCTYPE html>
<html>
<head>
    <title>Axios CRUD Example</title>
    <script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>
<body>
<script>
// Base URL of JSON Server
const BASE_URL = "http://localhost:3000/students";
// Reusable Axios function
function axiosRequest(method, url, data = null) {
    return axios({
        method: method,
        url: url,
        data: data,
        headers: { "Content-Type": "application/json" }
    })
    .then(response => response.data)
    .catch(error => {
        console.error("HTTP Error:", error);
        throw error;
    });
}

// GET all students
function getAllStudents() {
    return axiosRequest("GET", BASE_URL)
        .then(data => console.log("GET All:", data));
}
// GET student by ID
function getStudentById(id) {
    return axiosRequest("GET", `${BASE_URL}/${id}`)
        .then(data => console.log("GET One:", data));
}
// POST - add new student
function addStudent() {
```

```
    return axiosRequest("POST", BASE_URL, {
        name: "Aman",
        score: 78
    })
    .then(data => console.log("POST Added:", data));
}
// PUT - replace entire student data
function replaceStudent() {
    return axiosRequest("PUT", `${BASE_URL}/1`, {
        name: "Shubham Updated",
        score: 95
    })
    .then(data => console.log("PUT Updated:", data));
}
// PATCH - update only score field
function updateStudentScore() {
    return axiosRequest("PATCH", `${BASE_URL}/2`, {
        score: 92
    })
    .then(data => console.log("PATCH Updated:", data));
}
// DELETE - remove student
function deleteStudent() {
    return axiosRequest("DELETE", `${BASE_URL}/3`)
        .then(() => console.log("DELETE Done"));
}
// Execute all requests one by one
async function runAllRequestsAxios() {
    await getAllStudents();
    await getStudentById(1);
    await addStudent();
    await replaceStudent();
    await updateStudentScore();
    await deleteStudent();
}
runAllRequestsAxios();

</script>
</body>
</html>
```